

pamica: GPU-accelerated Adaptive Mixture Independent Component Analysis in Python with Fortran parity

Seyed Yahya Shirazi¹, Arnaud Delorme^{1,2}, and Scott Makeig¹

¹ Swartz Center for Computational Neuroscience, Institute for Neural Computation, University of California San Diego, USA ² Centre de Recherche Cerveau et Cognition (CerCo), CNRS, University of Toulouse, France ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Independent Component Analysis (ICA) separates electroencephalographic and magnetoencephalographic (EEG/MEG) recordings into maximally independent sources that isolate brain, muscle, and artifact activities for downstream analysis ([Iversen & Makeig, 2019](#); [Makeig et al., 1995](#); [Vigário et al., 1997](#)). Adaptive Mixture ICA (AMICA) ([Palmer et al., 2006, 2007, 2012](#)) models each source with a flexible, self-adjusting probability density and lets several ICA models coexist, one per segment of a recording. Among the algorithms benchmarked by Delorme et al. ([2012](#)) it recovers the least dependent and most dipolar decompositions, and therefore the most physiologically interpretable ones. Its reference implementation, by Jason Palmer, is a Fortran program distributed as a compiled binary callable from MATLAB/EEGLAB: awkward to install, restricted to the central processing unit (CPU), and unusable from Python.

pamica reproduces the reference Fortran results within numerical tolerance while running on the CPU, NVIDIA graphics processing units (GPUs, via CUDA), and Apple GPUs (through Apple's MLX array framework ([Hannun et al., 2023](#))). It is a complete reimplement on PyTorch ([Paszke et al., 2019](#)), NumPy ([Harris et al., 2020](#)), and SciPy ([Virtanen et al., 2020](#)) rather than a wrapper around the binary, and exposes a scikit-learn-style estimator under a BSD-3-Clause license. It writes the format EEGLAB's AMICA loader reads, so established MATLAB tooling consumes its results unchanged. The software is at <https://github.com/sccn/pAMICA> (archived at doi:10.5281/zenodo.21312148).

Statement of need

AMICA decompositions are well suited to equivalent-dipole source localization and automated component classification ([Pion-Tonachini et al., 2019](#)). Yet the reference implementation is MATLAB-only Fortran, an increasing obstacle as neuroimaging analysis moves toward Python, for example MNE-Python ([Gramfort et al., 2013](#)): modern pipelines need an AMICA that runs natively in Python and on a GPU, and that is validated to reproduce the Fortran reference numerically.

General-purpose Python ICA implementations do not fill this gap. scikit-learn and MNE-Python provide FastICA ([Hyvärinen & Oja, 2000](#)) and Infomax ([Bell & Sejnowski, 1995](#); [Lee et al., 1999](#)), while Picard ([Ablin et al., 2018](#)) offers faster-converging maximum-likelihood ICA; none implement AMICA's mixture of models, adaptive generalized-Gaussian densities, or Newton updates, so none can reproduce its decompositions. pamica is for analysts who want AMICA-quality decompositions in Python, for anyone with a GPU who wants faster runs than the CPU-only binary, and for methodologists who need a transparent reference to build on.

41 Validation to date is on EEG; the algorithm is modality-agnostic but MEG is untested.

42 State of the field

43 pamica complements rather than replaces the reference Fortran AMICA used with EEGLAB
44 (Delorme & Makeig, 2004): it keeps the same output format, adds a Python API with GPU
45 support, and can run the reference Fortran itself through a bundled dependency-free native
46 build. Three other Python AMICA reimplementations have appeared as of August 2026
47 (Esmaili, 2025; Herforth, 2026; Huberty, 2025), oriented toward MNE-Python; pamica adds
48 a scikit-learn-style array API, output in EEGLAB's exact on-disk layout, an MLX backend,
49 and an optional MNE-Python wrapper. Its Fortran-parity validation goes further than theirs:
50 score functions exact to floating-point resolution against the literal Fortran expressions, and a
51 distributional framework for the non-identifiable multi-model case.

52 Software design

53 The governing decision was to treat numerical parity with Palmer's Fortran, rather than
54 convergence to some independent solution, as the definition of correctness. The rest follows
55 from it. Wrapping the binary would have secured parity for free but inherited its CPU-only,
56 MATLAB-facing design; reimplementing put parity at risk. pamica does both, porting the
57 algorithm natively and also shipping the reference binary as a runnable engine, so users
58 can reproduce the parity claims on their own data and hardware. For the same reason the
59 port follows the reference's natural-gradient (Amari, 1998) expectation-maximization (EM)
60 formulation rather than an automatic-differentiation optimizer: an Adam/autograd backend
61 was written early and then deleted, because it converged to different optima and made the
62 name "AMICA" ambiguous. The port covers exact-EM mixture updates, a positive-definite
63 Newton step (Palmer et al., 2008), symmetric sphering, the five source-density families, a
64 mixture of ICA models, component sharing, and the mutual-information metrics used to score
65 separation quality (Frank et al., 2023).

66 Three array backends (PyTorch, MLX, NumPy) implement the same algorithm behind a
67 common estimator API, with the reference Fortran a fourth. The duplication is deliberate:
68 NumPy stays readable as an executable specification, and MLX exists because it is the more
69 efficient path on Apple GPUs: PyTorch's Metal backend trails MLX across the entire block-
70 size sweep, and at pamica's shipped defaults trails even the CPU it runs beside (see the
71 documentation). Double precision is the default because parity demands it; single precision is
72 available for Apple GPUs, which have no float64.

73 Validation

74 Parity is measured two ways: by Hungarian-matched component correlation, and by the
75 Amari distance (Amari et al., 1996), a relabeling- and scale-invariant metric that needs no
76 assignment step. Both implementations ran AMICA's default 2000 iterations with Newton
77 disabled (pamica's own default), to isolate the algorithm from initialization. With Newton
78 enabled and independent seeds, some of the weakest components settle into a different basin
79 of equal or higher likelihood: on one seed of three this affected ten of seventy components,
80 while the other two matched the reference at ~ 0.99 . A matched initialization restores ~ 0.997 ,
81 so this is a property of the initialization, not a parity defect. The single-model comparison uses
82 a well-determined external recording (NEMAR on002718, $k \approx 153$, where k = frames over
83 squared channel count (Frank et al., 2025)) alongside the bundled 32-channel sample ($k \approx 30$).
84 A mixture of ICA models is not partition-identifiable, so exact partition parity is the wrong
85 bar there; it is judged instead by whether the implementations sample a similar distribution of

solutions, over ensembles of 20 runs each (Figure 1). A permutation test finds no evidence that cross-implementation agreement is worse than Fortran's own run-to-run agreement.

Table 1: Parity of pamica with the Fortran reference. Multi-model rows are over 20-run ensembles (190 within-, 400 cross-implementation pairs); sd is the standard deviation, given where computed. The final row is the one metric on which the ensembles differ significantly, and it is convergence speed rather than optimum quality: pamica reaches Fortran's mean by 200 iterations and passes it by 300.

Regime	Metric (dataset)	Result (mean)
Single	Log-likelihood gap (on002718)	within ~ 0.0005 of -3.6993
Single	Component correlation (on002718)	0.998
Single	Amari distance (bundled)	0.006
Single	Score functions, sufficient statistics	exact, $\sim 10^{-15}$
Multi	Correlation, one run: cross; within-Fortran	0.65; 0.64 (sd 0.05)
Multi	Amari, one run: cross; within-Fortran	0.163; 0.174 (sd 0.02)
Multi	Ensemble agreement, cross — within-Fortran	correlation $+0.011$ ($p = 0.96$); Amari -0.011 ($p > 0.999$)
Multi	Ensemble log-likelihood: Fortran; pamica	-3.3539 ; -3.3629 (Kolmogorov-Smirnov $p = 6 \times 10^{-5}$)

Multi-model AMICA (n_models=2): pamica vs Fortran ensembles, real sample EEG

Dashed vertical lines mark each distribution's mean.

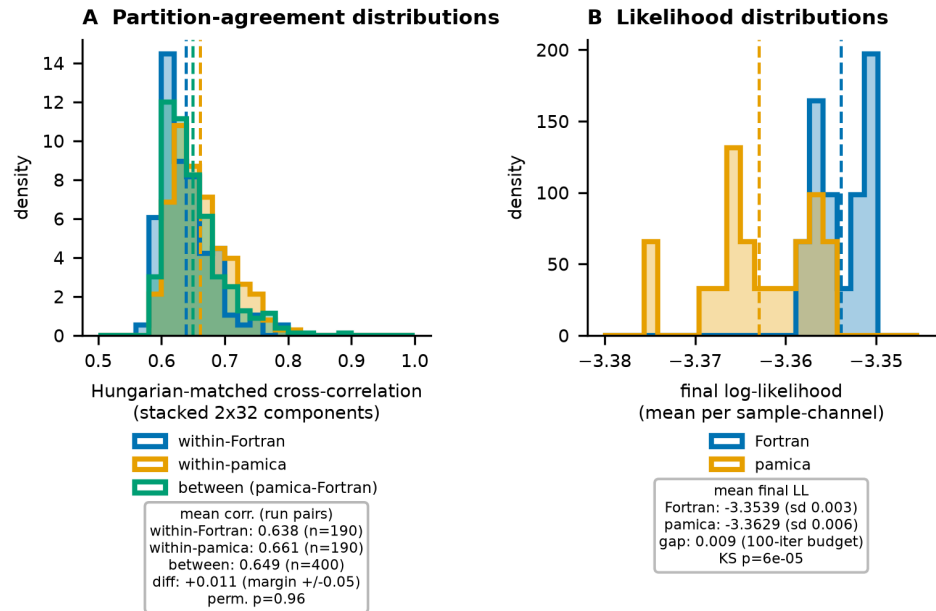


Figure 1: Multi-model ensemble partition-correlation (A) and log-likelihood (B) distributions, 20 pamica and 20 Fortran fits of the sample EEG; dashed lines mark each mean. A's three distributions overlap, so the single-run correlation reflects intrinsic run-to-run spread, not a gap to the reference. B's separation is Table 1's 0.009 gap on a ~ 0.035 axis.

88 All backends converge to the same single-model log-likelihood on real EEG (maximum pairwise
89 difference ~ 0.003), and single precision matches double to four or five significant digits on
90 that log-likelihood. Component-level float32 agreement is not yet characterized at a matched
91 iteration budget, so float64 remains the default for parity work, and double-precision CUDA is
92 the reproducible NVIDIA path.

93 On real 70-channel EEG at block_size=512, per-iteration cost is 25 ms for MLX on an Apple
94 GPU, 39 ms for double-precision CUDA on an RTX 4090, and 30 ms for native Fortran
95 on a 24-core i9-13900K, against 193 ms for PyTorch on an Apple-Silicon CPU and 255 ms
96 for PyTorch-Metal. That comparison is block-size-confounded: PyTorch-Metal is far more
97 block-size-sensitive than the CPU it runs beside, falling from 431 to 30.5 ms/iteration on
98 the bundled 32-channel sample between block_size=512 and pamica's current 8192 default,
99 still behind the CPU's 21.7 ms/iteration there, and to 13.5 ms/iteration, below the CPU's
100 15.8 ms, at a further-tuned single-block setting that is memory-limited rather than a free win,
101 since peak block memory scales with block_size, which is why 8192 stays the shipped default.
102 MLX stays the fastest Apple backend throughout the sweep. Full tables, the data-size sweep,
103 and reproduction commands are in the [documentation](#); the correctness harness never uses
104 synthetic data. The harness and sample data ship with the package, and every row of Table 1
105 but the two on002718 entries re-runs from the bundled sample alone.

106 Research impact statement

107 AMICA is not a new method: its source model and estimation were published in 2006-2008
108 (Palmer et al., 2006, 2007, 2008), and it ranked first of the twenty-two algorithms benchmarked
109 by Delorme et al. (2012). MNE-Python is publicly weighing which of the Python ports to

110 adopt (mne-tools/mne-python issue 13819), where an MNE core developer and the author of
111 a competing port lean toward *pamica*, since this Center develops AMICA and maintains the
112 reference Fortran and EEGLAB.

113 sccn/pAMICA has been public since 2021, with public issues, pull requests, seven releases (four
114 on PyPI), and a Zenodo archive. Three researchers outside the author group have filed issues,
115 two mid-analysis on their own data: a 236-channel, 8.3-million-sample EEG decomposition
116 (issue 207) and Maxwell-filtered 306-channel MEG (issue 221); the third is adopting AMICA
117 for a multiverse analysis in MNE-Python (issue 206). Integration is underway, not complete,
118 into this Center's Python preprocessing and into NEMAR (Delorme et al., 2022), the public
119 archive we operate: 800 datasets, ~40,000 participants, 55 TB.

120 AI usage disclosure

121 Generative AI was used in this project, disclosed here under the journal's policy.

Tools. Anthropic's Claude models (Sonnet and Opus families), through the Claude Code command-line assistant. The instructions given to them are public in the repository (AGENTS.md, CLAUDE.md, .rules/).

125 **Scope.** The source code (translating the reference Fortran into Python, refactoring, scaffolding
126 tests), the documentation, and the drafting and copy-editing of this manuscript.

Human oversight. The authors made the design decisions and take responsibility for the result. Parity as the correctness criterion, the natural-gradient EM formulation, the backend architecture, the distributional treatment of the multi-model case, and the acceptance thresholds were chosen by the authors, not by a model. Every AI-assisted change was reviewed by a human before merge and run through the parity harness, which scores output against the reference binary on real recordings rather than generated fixtures. The reported numbers came from running the software and were checked against their run records, as was every bibliographic entry.

135 Acknowledgments

We thank Jason Palmer and his advisor Ken Kreutz-Delgado, co-developers of AMICA, for the reference implementation, and the EEGLAB community for the tools and sample data used to validate this work. Two authors developed the methods pamica builds on: S.M. co-developed AMICA (Palmer et al., 2012) and A.D. is a lead developer of EEGLAB (Delorme & Makeig, 2004). This work was supported by The Swartz Foundation (Old Field, NY) to the Swartz Center for Computational Neuroscience and by National Institutes of Health grant R01-NS047293 (to A.D. and S.M.).

143 References

- Ablin, P., Cardoso, J.-F., & Gramfort, A. (2018). Faster independent component analysis by preconditioning with hessian approximations. *IEEE Transactions on Signal Processing*, 66(15), 4040–4053. <https://doi.org/10.1109/TSP.2018.2844203>
- Amari, S.-I. (1998). Natural gradient works efficiently in learning. *Neural Computation*, 10(2), 251–276. <https://doi.org/10.1162/089976698300017746>
- Amari, S.-I., Cichocki, A., & Yang, H. H. (1996). A new learning algorithm for blind signal separation. *Advances in Neural Information Processing Systems*, 8, 757–763.
- Bell, A. J., & Sejnowski, T. J. (1995). An information-maximization approach to blind separation and blind deconvolution. *Neural Computation*, 7(6), 1129–1159. <https://doi.org/10.1162/neco.1995.7.6.1129>

- 153 [org/10.1162/neco.1995.7.6.1129](https://doi.org/10.1162/neco.1995.7.6.1129)
- 154 Delorme, A., & Makeig, S. (2004). EEGLAB: An open source toolbox for analysis of single-
155 trial EEG dynamics including independent component analysis. *Journal of Neuroscience*
156 *Methods*, 134(1), 9–21. <https://doi.org/10.1016/j.jneumeth.2003.10.009>
- 157 Delorme, A., Palmer, J., Onton, J., Oostenveld, R., & Makeig, S. (2012). Independent EEG
158 sources are dipolar. *PLoS ONE*, 7(2), e30135. [https://doi.org/10.1371/journal.pone.](https://doi.org/10.1371/journal.pone.0030135)
159 [0030135](https://doi.org/10.1371/journal.pone.0030135)
- 160 Delorme, A., Truong, D., Youn, C., Sivagnanam, S., Stirn, K., Yoshimoto, K., Poldrack,
161 R. A., Majumdar, A., & Makeig, S. (2022). NEMAR: An open access data, tools and
162 compute resource operating on neuroelectromagnetic data. *Database*, 2022, baac096.
163 <https://doi.org/10.1093/database/baac096>
- 164 Esmaili, S. (2025). *Amica: Native python implementation of AMICA for MNE-Python and*
165 *scientific EEG workflows*. <https://github.com/snesmaeli/amica>.
- 166 Frank, G., Shirazi, S. Y., Palmer, J., Cauwenberghs, G., Makeig, S., & Delorme, A. (2023).
167 An exploration of optimal parameters for efficient blind source separation of EEG
168 recordings using AMICA. *2023 IEEE 23rd International Conference on Bioinformatics and*
169 *Bioengineering (BIBE)*, 205–210. <https://doi.org/10.1109/BIBE60311.2023.00040>
- 170 Frank, G., Shirazi, S. Y., Palmer, J., Cauwenberghs, G., Makeig, S., & Delorme, A. (2025).
171 Quantitative exploration of sufficient data quantities for EEG decomposition using AMICA.
172 *2025 12th International IEEE/EMBS Conference on Neural Engineering (NER)*, 532–536.
173 <https://doi.org/10.1109/NER61569.2025.11588993>
- 174 Gramfort, A., Luessi, M., Larson, E., Engemann, D. A., Strohmeier, D., Brodbeck, C., Goj,
175 R., Jas, M., Brooks, T., Parkkonen, L., & Hämäläinen, M. (2013). MEG and EEG data
176 analysis with MNE-Python. *Frontiers in Neuroscience*, 7, 267. <https://doi.org/10.3389/fnins.2013.00267>
- 177
- 178 Hannun, A., Digani, J., Katharopoulos, A., & Collobert, R. (2023). *MLX: Efficient and flexible*
179 *machine learning on apple silicon*. <https://github.com/ml-explore/mlx>.
- 180 Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D.,
181 Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming with
182 NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- 183 Herforth, J. (2026). *Pyamica: PyTorch AMICA, adaptive mixture independent component*
184 *analysis*. <https://github.com/DerAndereJohannes/pyamica>.
- 185 Huberty, S. (2025). *Amica-python: Python implementation of adaptive mixture ICA*. <https://github.com/scott-huberty/amica-python>.
- 186
- 187 Hyvärinen, A., & Oja, E. (2000). Independent component analysis: Algorithms and applications.
188 *Neural Networks*, 13(4-5), 411–430. [https://doi.org/10.1016/S0893-6080\(00\)00026-5](https://doi.org/10.1016/S0893-6080(00)00026-5)
- 189 Iversen, J. R., & Makeig, S. (2019). MEG/EEG data analysis using EEGLAB. In
190 *Magnetoencephalography: From signals to dynamic cortical networks* (pp. 391–406).
191 Springer International Publishing. https://doi.org/10.1007/978-3-319-62657-4_8-1
- 192 Lee, T.-W., Girolami, M., & Sejnowski, T. J. (1999). Independent component analysis using
193 an extended infomax algorithm for mixed subgaussian and supergaussian sources. *Neural*
194 *Computation*, 11(2), 417–441. <https://doi.org/10.1162/089976699300016719>
- 195 Makeig, S., Bell, A. J., Jung, T.-P., & Sejnowski, T. J. (1995). Independent component
196 analysis of electroencephalographic data. *Advances in Neural Information Processing*
197 *Systems*, 8, 145–151.
- 198 Palmer, J. A., Kreutz-Delgado, K., & Makeig, S. (2006). Super-Gaussian mixture source model
199 for ICA. *Independent Component Analysis and Blind Signal Separation: Proceedings of the*

- 200 6th International Conference (ICA 2006), 854–861. https://doi.org/10.1007/11679363_106
- 201 Palmer, J. A., Kreutz-Delgado, K., & Makeig, S. (2012). *AMICA: An adaptive*
202 *mixture of independent component analyzers with shared components*. Swartz
203 Center for Computational Neuroscience, University of California San Diego.
204 https://sccn.ucsd.edu/~jason/amica_a.pdf
- 205 Palmer, J. A., Kreutz-Delgado, K., Rao, B. D., & Makeig, S. (2007). Modeling and estimation
206 of dependent subspaces with non-radially symmetric and skewed densities. In M. E. Davies,
207 C. J. James, S. A. Abdallah, & M. D. Plumbley (Eds.), *Independent component analysis*
208 *and signal separation: Proceedings of the 7th international conference (ICA 2007)* (pp.
209 97–104). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-540-74494-8_13
- 210 Palmer, J. A., Kreutz-Delgado, K., Rao, B. D., & Makeig, S. (2008). Newton method for the
211 ICA mixture model. *2008 IEEE International Conference on Acoustics, Speech and Signal*
212 *Processing (ICASSP)*, 1805–1808. <https://doi.org/10.1109/ICASSP.2008.4517982>
- 213 Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin,
214 Z., Gimelshein, N., Antiga, L., & others. (2019). PyTorch: An imperative style, high-
215 performance deep learning library. *Advances in Neural Information Processing Systems*, 32,
216 8024–8035.
- 217 Pion-Tonachini, L., Kreutz-Delgado, K., & Makeig, S. (2019). ICLabel: An automated
218 electroencephalographic independent component classifier, dataset, and website.
219 *NeuroImage*, 198, 181–197. <https://doi.org/10.1016/j.neuroimage.2019.05.026>
- 220 Vigário, R., Jousmäki, V., Härmäläinen, M., Hari, R., & Oja, E. (1997). Independent component
221 analysis for identification of artifacts in magnetoencephalographic recordings. *Advances in*
222 *Neural Information Processing Systems*, 10, 229–235.
- 223 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,
224 Burovski, E., Peterson, P., Weckesser, W., Bright, J., & others. (2020). SciPy 1.0:
225 Fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3),
226 261–272. <https://doi.org/10.1038/s41592-019-0686-2>